

DAT410 - Design of AI Systems

Group 69, Module 1		
Teammates	Jana Leong	Chung Fung Oi
Course	Bachelor of Computer Science	
Email	janale@student.chalmers.se	funco@student.chalmers.se
No. of hours spent	15	

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part in other solutions.

1. Reading and reflection

Read the papers "[The Netflix Prize](#)" and "[Lessons from the Netflix Prize Challenge](#)" and write a half-page summary of your take-aways from them.

Pick two design features (e.g., regarding data, models, algorithms) that characterise the task and the solution in the papers and discuss how they may differ in another application of recommendation systems.

Key Summaries & Takeaways:

The Netflix Prize Challenge demonstrated the power of large-scale, real-world data and open competition between people in order to advance recommender systems. Netflix released a dataset with over 100 million anonymized movie ratings and challenged various communities to develop systems that could beat its inhouse Cinematch system based on the root mean square error (RSME) of the system's prediction against the actual rating that a subscriber provides.

The competition revealed that relatively simple baselines (e.g. user ratings) already capture meaningful signals (information), however to gain meaningful knowledge in order to recommend a new movie to a user would require models that were more holistic. Meaningful gains required models to exploit latent structure in user-item interactions, particularly matrix factorization approaches such as SVD. Over time, progress slows, illustrating diminishing returns and the difficulty of squeezing out the last few percentage points of accuracy.

No single model was ultimately the answer to the problem but rather the ensemble of diverse complementary predictor models that help each other in equal but opposite ways, addressing a different aspect of the data. It shows that there is a need to carefully regularize data, cater for changes in user preferences over time and engineer sensitivity to overfitting. Often review data does not come in rich format (specific useful information) but rather a bunch of information with much random noise and one who designs models have to be well aware in order to engineer a well done solution.

Design Feature 1: "Binary View" — In order to find out which movies users will rate, regardless of how they rated the movies, define a tool to find out which movies users will choose to voice their rating for (a rating could be positive or negative). This enables us to garner more information e.g. users will often rate movies selectively, only rating films they feel strongly about making data sparse, affecting many models that rely on more data. The binary view enables us to bridge the gap by providing insight to movies that a user is more likely to engage with, even if they don't give rating directly. In the same vein (of dealing with sparse data), traditional models find it difficult to model over users who have not rated enough movies. This view therefore finds its way to build a richer profile that a user might be interested in based on their general interest patterns, even if they haven't rated enough items.

E.g. The user watched movie A and rated it good, watched movie B and rated it bad, watched movie C with no rating given, watched movie D halfway and no rating was given (did not come

back to finish it within a significant time). We glean more information as now we have more than just 1 factor of an explicit good or bad rating but rather how the user is likely to arrange and highlight movies. For example, the user might forget about D, only remember the name of movie C but not its story, could tell you the story in detail about movie A and hated movie B and could tell you all about why they hated it. All these implicit behaviours (engagement with movie A and B) can be recorded in this binary view of data. Hence, proving it rather useful in an explicit ratings dataset, as it provides more information for traditional methods to build upon.

Furthermore, in many real world systems, there are many implicit behaviors that aren't captured explicitly. For example, users might browse, rent or watch movies without leaving a rating. By capturing whether a user chose to rate a movie, the system gets an extra signal about a user's preferences than the absence of a rating alone.

In other recommendation systems, such as content-based systems (news, job recommendations), the binary view might be less relevant, as clicks, searches or views replace direct ratings. Implicit data like browsing history or dwell time would serve as a better signal for preference modeling.

Overall, the binary view highlights the potential of implicit feedback in enriching models based on explicit ratings, which is highly valuable in real-world recommender systems.

Design Feature 2: "Matrix Factorization (SVD) and Ensemble Methods" — Matrix factorization techniques, such as SVD, were critical to the success of many of the top models in the Netflix Prize competition. In matrix factorization, the large matrix of user-items ratings is decomposed into latent factors, capturing the underlying patterns in user preferences and item characteristics. This technique is especially useful in collaborative filtering, where the goal is to predict ratings based on past interactions without needing explicit content information (like movie genres).

How SVD works in this context:

- The user-item matrix is very sparse, meaning there are a lot of missing ratings (most users only rate a small fraction of movies). SVD helps us to reduce the matrix to a lower-dimensional representation, making it easier to identify patterns in user behaviour and item popularity.
- Each user and item are represented by a set of latent factors e.g. hidden features like "action preferences" for users or "violence level" for movies. That helps predict unknown ratings by modeling how different latent factors interact.
- Singular Value Decomposition (SVD), specifically regularized SVD, was one of the most powerful techniques used in the Netflix Prize competition, as it allowed for dimensionality reduction while capturing key aspects of user-item interactions.

Integration with Ensemble Methods:

- Ensembling became a key part of the Netflix Prize winning solution. Individual models, such as SVD, were not sufficient on their own, and top teams combined several models to create more robust predictions.
- For example, a “blending” strategy was used, where predictions from multiple models are combined together to achieve better performance. This allowed the models to correct each other’s mistakes, improving overall accuracy.
- Ensemble methods could involve simple averaging or more complex approaches like stacking, where the outputs of multiple models were used as input features for another model.

However, in content-based systems (e.g. job recommendations) matrix factorization techniques like SVD might not be as directly applicable because the focus is on the content (features of the items such as job description, movie plots) rather than the relationships between user and items. Instead techniques like TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings might be useful to model item similarities based on textual data. TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (or corpus). It considers how often a word appears in a document (its term frequency) but adjusts for how common the word is across all documents (the inverse document frequency) (think used in phone’s auto-complete technologies). In the context of content-based systems, we can use TF-IDF to model the importance of specific words (e.g. “action”, “comedy”) in movie descriptions. And if you are interested in action movies then the system will recommend movies with a similar set of keywords based on their TF-IDF score. Ensemble methods could still be used to combine predictions from different content-based models.

In implicit feedback based systems (e.g. news recommenders), a traditional matrix factorization model might not be as effective because implicit feedback is not as reliable as explicit ratings for learning user preferences. However, matrix factorization can be adapted to work with implicit data by using methods like Alternating Least Squares (ALS). ALS is a matrix factorization method commonly used for collaborative filtering in recommendation systems. It decomposes the user-item interaction into 2 smaller matrices (user factors & item factors), and the model alternates between fixing one matrix and solving for the other. In this case the model would be designed to predict the probability of a user interacting with an item rather than their rating. It can be used alongside SVD or in addition to SVD. Particularly useful when dealing with implicit feedback, like play time, views, clicks or purchase history where we do not have an explicit score. ALS also handles user-item interaction matrices efficiently, particularly for large datasets like Netflix’s, furthermore it is inexpensive computationally. Therefore, ensemble models still play a crucial role here, combining different types of implicit signals to build more accurate user profiles.

2. Implementation

- The collection [movie_reviews.zip](#) contains a dataset of movie reviews.
- In *user_reviews.csv*, rows represent users of the system and columns products. The cells represent a numerical rating from 1 to 5 of a product given by a user. **A value of 0 represents a missing rating—a movie the user hasn't rated yet.**
- In *movie_genres.csv*, the rows represent the movies in the columns of *user_reviews.csv*. Here, each row represents genre features of a single movie.
- **Your task is to suggest which movies users should be recommended next that they haven't seen before.**
- You may use any suitable method to develop a recommendation system based on the data given in the data files. You are not required to use all data.
- **In your report, describe the system, motivate the choices you made. Comment on the strength and the weaknesses of the system. Suggest 5 movies for each of the first 5 users, Vincent, Edgar, Addilyn, Marlee and Javier, that they haven't rated already.**

When approaching this problem, the initial approach we considered was a content-based solution using cosine similarity. This would compare movies based on their genres. However, we realised this approach would only capture genre-specific preferences and miss the advantages of collaborative filtering — the ability to recommend movies based on the similar tastes of other users. Collaborative filtering can leverage the shared preferences of users to improve recommendations by identifying patterns beyond individual preferences.

To incorporate this, we pivoted to using a collaborative filtering model using Singular Value Decomposition (SVD) from the Surprise library. SVD is effective in capturing hidden patterns in user preferences based on their rating behaviours, making it ideal for this task. By decomposing the user-item interaction matrix, SVD can help reveal latent factors, such as a user's affinity for action films or the level of intensity from a movie, enabling personalised recommendations even for unrated items.

The success of SVD in creating recommendation systems was highlighted by its use in the Netflix Prize Challenge. The competition demonstrated that matrix factorization techniques like SVD outperformed simpler models in handling large, sparse datasets. SVD excels at capturing hidden relationships in sparse data, which makes it particularly useful in recommendation systems where data is often incomplete.

However, after testing the SVD model, we encountered popularity bias. The model tended to recommend popular movies across users, neglecting individual preferences. This bias is a well-known issue in collaborative filtering, especially in sparse datasets. To address this, we developed a hybrid recommendation system, combining collaborative filtering and content based filtering. This hybrid approach effectively reduces popularity bias while maintaining accuracy (RMSE $\approx$ 1.16). Our testing also revealed that while pure SVD caused 4-5 movie overlaps between users, the hybrid model reduced the overlap to 1 to 3 movies, showcasing that it provides more personal recommendations. This mirrors the findings from the Netflix Prize

that methods combining different models yield the best results. This method also handles sparse data effectively. With only ~5% rating density, SVD alone might struggle to make meaningful recommendations for users with sparse profiles. However, by incorporating genre similarity, the hybrid system compensates for missing ratings, allowing recommendations for users with few ratings.

However, this hybrid system has notable weaknesses. The most significant of them is in regards to new users. The system faces a critical limitation for new users with no rating history, as it struggles to generate personalized recommendations. While content-based filtering can recommend unrated movies based on genre or other content features, the system needs user feedback to refine the recommendations and address the cold start problem effectively. This is a critical limitation for platforms that are still growing and acquiring new users.

Another weakness would be the computational cost for a large dataset. The genre similarity matrix requires significant memory ($O(n^2)$ space, which becomes computationally expensive for large datasets like Netflix's. Additionally, calculating genre similarity for all unrated movies for each user adds significant overhead compared to using pure SVD, which operates directly on the sparse user-item matrix and does not require the genre-based calculations.

Lastly, the system's recommendations are influenced by the user's past ratings and genre choice. If a user's ratings primarily involve one genre, the system will favor other movies in the same genre, potentially limiting the diversity of recommendations.

Our code:

```
# Load both datasets
reviews_df = pd.read_csv('user_reviews.csv', header=0)
if "Unnamed: 0" in reviews_df.columns:
    reviews_df.drop(columns=['Unnamed: 0'], inplace=True)
genres_df = pd.read_csv('movie_genres.csv', header=0)
if "Unnamed: 0" in genres_df.columns:
    genres_df.drop(columns=['Unnamed: 0'], inplace=True)

# Transform data from wide to long format for Surprise library
# Melt the dataframe: keep 'User' column, convert all movie columns to rows
ratings_long = reviews_df.melt(id_vars=['User'],
                                var_name='Movie',
                                value_name='Rating')

# Remove rows with 0.0 rating as SVD requires actual ratings (1-5) to learn patterns
ratings_long = ratings_long[ratings_long['Rating'] > 0.0]
# Reset index after filtering
ratings_long = ratings_long.reset_index(drop=True)
```

```

# Create a Reader object with the rating scale
reader = Reader(rating_scale=(1, 5))
# Load data into Surprise Dataset format
data = Dataset.load_from_df(ratings_long[['User', 'Movie', 'Rating']], reader)
# Build full trainset
trainset = data.build_full_trainset()
# Create and train the SVD model
algo = SVD(n_factors=50, n_epochs=20, lr_all=0.005, reg_all=0.02, random_state=42)
algo.fit(trainset)

def get_recommendations(user_name, n_recommendations=5):
    """
    Get top N movie recommendations for a specific user using Surprise

    Parameters:
    - user_name: Name of the user
    - n_recommendations: Number of recommendations to return

    Returns:
    - DataFrame with recommended movies and predicted ratings
    """
    # Get all movies
    all_movies = reviews_df.columns[1:].tolist()

    # Get movies the user has already rated
    user_ratings = reviews_df[reviews_df['User'] == user_name].iloc[0, 1:]
    rated_movies = user_ratings[user_ratings > 0].index.tolist()

    # Get unrated movies
    unrated_movies = [movie for movie in all_movies if movie not in rated_movies]

    # Predict ratings for all unrated movies
    predictions = []
    for movie in unrated_movies:
        pred = algo.predict(user_name, movie)
        predictions.append({
            'Movie': movie,
            'Predicted_Rating': pred.est
        })

    # Sort by predicted rating and get top N
    predictions_df = pd.DataFrame(predictions)

```

```

top_recommendations = predictions_df.nlargest(n_recommendations, 'Predicted_Rating')

# Calculate user statistics
n_rated = len(rated_movies)
avg_rating = user_ratings[user_ratings > 0].mean()

print(f"\n{'='*70}")
print(f"Recommendations for: {user_name}")
print(f"{'='*70}")
print(f"User has rated {n_rated} movies with average rating: {avg_rating:.2f}")
print(f"\nTop {n_recommendations} Recommended Movies:\n")

return top_recommendations.reset_index(drop=True)

def get_recommendations_hybrid(user_name, n_recommendations=5, collab_weight=0.7,
content_weight=0.3):
    """
    HYBRID RECOMMENDER: Collaborative Filtering + Content-Based Filtering

    Combines:
    1. SVD predictions (what similar users rated highly)
    2. Genre similarity (what genres the user actually liked)

    Parameters:
    - user_name: Name of the user
    - n_recommendations: Number of recommendations to return
    - collab_weight: Weight for collaborative filtering score (0-1)
    - content_weight: Weight for content-based score (0-1)
    """

    all_movies_list = reviews_df.columns[1:].tolist()
    user_ratings = reviews_df[reviews_df['User'] == user_name].iloc[0, 1:]
    rated_movies = user_ratings[user_ratings > 0].index.tolist()
    unrated_movies = [movie for movie in all_movies_list if movie not in rated_movies]

    # Step 1: Get collaborative filtering scores (SVD predictions)
    collab_scores = {}
    for movie in unrated_movies:
        pred = algo.predict(user_name, movie)
        collab_scores[movie] = pred.est

    # Step 2: Get content-based scores (genre similarity)
    # Find movies the user rated highly (4+)

```



```

high_rated_indices = [all_movies_list.index(m) for m in rated_movies
                       if user_ratings[m] >= 4.0]

content_scores = {}
for unrated_movie in unrated_movies:
    unrated_idx = all_movies_list.index(unrated_movie)

    # Average similarity to all highly-rated movies
    if high_rated_indices:
        similarities = [genre_similarity[unrated_idx][hr_idx] for hr_idx in
                        high_rated_indices]
        avg_similarity = np.mean(similarities)
    else:
        # If no highly-rated movies, use average similarity to all rated movies
        rated_indices = [all_movies_list.index(m) for m in rated_movies]
        similarities = [genre_similarity[unrated_idx][r_idx] for r_idx in
                        rated_indices]
        avg_similarity = np.mean(similarities) if rated_indices else 0.5

    content_scores[unrated_movie] = avg_similarity

# Step 3: Combine scores (weighted average)
# Normalize both scores to 0-5 scale for fair comparison
collab_min, collab_max = min(collab_scores.values()), max(collab_scores.values())
content_min, content_max = 0, 1

hybrid_scores = {}
for movie in unrated_movies:
    # Normalize to 1-5 scale
    norm_collab = 1 + (collab_scores[movie] - collab_min) / (collab_max - collab_min
                                                             + 0.001) * 4
    norm_content = 1 + content_scores[movie] * 4

    # Weighted combination
    hybrid_scores[movie] = collab_weight * norm_collab + content_weight *
                           norm_content

# Sort and get top N
top_movies = sorted(hybrid_scores.items(), key=lambda x: x[1], reverse=True)
[:n_recommendations]

# Create results dataframe
results = pd.DataFrame({

```

```

        'Movie': [m[0] for m in top_movies],
        'Hybrid_Score': [m[1] for m in top_movies],
        'Collab_Score': [collab_scores[m[0]] for m in top_movies],
        'Genre_Score': [content_scores[m[0]] for m in top_movies]
    })

    # User stats
    n Rated = len(rated_movies)
    avg_rating = user_ratings[user_ratings > 0].mean()

    print(f"\n{' '*70}")
    print(f"HYBRID RECOMMENDATIONS for: {user_name}")
    print(f"{' '*70}")
    print(f"User has rated {n Rated} movies | Average rating: {avg_rating:.2f}")
    print(f"Recommendation weights: {collab_weight*100:.0f}% Collaborative +
          {content_weight*100:.0f}% Genre-Based\n")

    return results

# Generate recommendations for the 5 specified users
target_users = ["Vincent", "Edgar", "Addilyn", "Marlee", "Javier"]

all_recommendations = {}

for user in target_users:
    recommendations = get_recommendations(user, n_recommendations=5)
    all_recommendations[user] = recommendations
    display(recommendations)
    print("\n")

```

Our final results:

```
=====
Recommendations for: Vincent
=====
User has rated 39 movies with average rating: 3.82

Top 5 Recommended Movies:
```

	Movie	Predicted_Rating
0	Mad City	4.613834
1	The Edge	4.576289
2	Perrier's Bounty	4.523202
3	Sinbad: Legend of the Seven Seas	4.503118
4	Now You See Me 2	4.438908

```
=====
Recommendations for: Edgar
=====
User has rated 30 movies with average rating: 3.87

Top 5 Recommended Movies:
```

	Movie	Predicted_Rating
0	Perrier's Bounty	4.873071
1	Seeking a Friend for the End of the World	4.678882
2	The Good Thief	4.595424
3	Maximum Risk	4.538663
4	Dysfunctional Friends	4.524821

```
=====
Recommendations for: Addilyn
=====
User has rated 36 movies with average rating: 3.72

Top 5 Recommended Movies:
```

	Movie	Predicted_Rating
0	Maximum Risk	4.464699
1	Perrier's Bounty	4.460838
2	Sinbad: Legend of the Seven Seas	4.315414
3	Seeking a Friend for the End of the World	4.308851
4	Now You See Me 2	4.290849

```
=====
Recommendations for: Marlee
=====
User has rated 32 movies with average rating: 3.47

Top 5 Recommended Movies:
```

	Movie	Predicted_Rating
0	Maximum Risk	4.297810
1	Perrier's Bounty	4.267678
2	The Hunting Party	4.204512
3	The Edge	4.154430
4	Mad City	4.146851

```
=====
Recommendations for: Javier
=====
User has rated 28 movies with average rating: 3.14

Top 5 Recommended Movies:
```

	Movie	Predicted_Rating
0	The Hunting Party	4.074182
1	Perrier's Bounty	4.022104
2	Sinbad: Legend of the Seven Seas	4.007510
3	Witness	3.969720
4	A Scanner Darkly	3.942137

3. Discussion

- Assessing the quality of a recommendation system before deploying it to users is difficult. Why? In a few paragraphs, discuss fundamental challenges in evaluation of recommendation systems and how they may lead to problems in practice.

Assessing the quality of a recommendation system before deployment presents several challenges, mainly due to the complexity of real-world user behaviour and the unpredictability of user interaction within a system. Evaluating recommendation systems in a realistic setting is a challenge as they are usually assessed & modelled using offline data. They are also evaluated based on metrics like accuracy and error rates. However, these metrics may not correlate well with user engagement in a real-world scenario. Users may interact with recommendations differently in practice, influenced by factors like context, trends and diversity, which is often overlooked in offline evaluations.

Another challenge would stem from the dynamic nature of user preferences over time. A recommendation system struggles when recommending items to users as people's taste and behaviours can change drastically year after year. Conversely, new users may not have enough data in the system to understand their preferences. This makes it difficult for the system to predict future preferences accurately. This variability can lead to issues where the system may continue recommending outdated content that no longer aligns with the user's current interest, or struggle to keep up with evolving trends. This makes it difficult to evaluate the system's effectiveness at scale, especially in the early stages of deployment when the system may not have a large enough data set to work with. Therefore, a recommendation system that relies solely on historical data may continue recommending outdated or irrelevant content, leading to poor user experiences. Evaluating a recommendation system in such a dynamic, real-time setting is challenging because offline metrics often fail to capture how well the system adapts to changing preferences and real-time user feedback.

Additionally, there is the challenge of bias in evaluation metrics. Common evaluation metrics like accuracy or RMSE often focus on how well the system predicts known preferences (e.g., ratings that users have already provided). However, these metrics may not capture other important aspects of recommendation systems, such as diversity, serendipity (unexpected but interesting suggestions), or fairness. Focusing solely on traditional accuracy metrics can result in systems that are overly focused on recommending popular or highly rated items, while neglecting niche or less mainstream content, which may negatively impact user engagement in the long term. The lack of engagement ultimately can result in user disengagement, where users are tired of seeing the same recommendations or feel that the system does not understand their evolving interests. Hence, relying solely on predictive accuracy can skew evaluations, leaving out factors that are crucial for long-term user satisfaction.

Finally, evaluation in live settings introduces real-time complexities such as the impact of feedback loops. Recommendation systems that are deployed in a live environment often adjust to user behavior over time, meaning that the recommendations they produce can influence the very preferences they are trying to predict. This creates a situation where the system learns

from its own predictions, potentially leading to overfitting or reinforcing existing biases. In practice this means that user engagement and preference trends might change in ways that the system cannot predict accurately if not accounted for during training. Thus, evaluating these systems before deployment, without accounting for these feedback loops, can lead to misleading conclusions about their effectiveness in real-world conditions.

These challenges make it essential to combine offline evaluations with more nuanced, real-time testing and consider a broader set of metrics that encompass both predictive accuracy and user satisfaction.